Java Introduction

What is Java?

- Java is a popular high-level, object-oriented programming language, which was originally developed by Sun Microsystems and released in 1995.
- Currently, Java is owned by Oracle and more than 3 billion devices run Java.
- Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.
- Today Java is being used to develop numerous types of software applications including Desktop Apps, Mobile apps, Web apps, Games, and much more.

Java Example

Let's have a quick look at Java programming example.

Simple.java

```
1. class Test{
2.     public static void main(String args[]){
3.      System.out.println("Hello Java");
4.     }
5. }
```

File -> Save -> d:\Test.java

• Compiling the program

- To compile the program, we must run the Java compiler (javac), with the name of the source file on “command prompt” like as follows
- If everything is OK, the “javac” compiler creates a file called “Test.class” containing byte code of the program.

• Running the program

- We need to use the Java Interpreter to run a program (Java Test).

Application

- According to Sun, 3 billion devices run Java.
- There are many devices where Java is currently used.
- Some of them are as follows:

1. Desktop Applications such as acrobat reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, javatpoint.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

Types of Java Applications

- There are mainly 4 types of applications that can be created using Java programming:

1) Standalone Application

- Standalone applications are also known as desktop applications or window-based applications.
- These are traditional software that we need to install on every machine.
- Examples of standalone application are Media player, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application

- An application that runs on the server side and creates a dynamic page is called a web application.
- Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF(Javaserver Faces), etc. technologies are used for creating web applications in Java.

3) Enterprise Application

- An application that is distributed in nature, such as banking applications, etc. is called an enterprise application.
- It has advantages like high-level security, load balancing, and clustering. In Java, EJB(Enterprise java beans) is used for creating enterprise applications.

4) Mobile Application

- An application which is created for mobile devices is called a mobile application.
- Currently, Android and Java ME(Micro Edition) are used for creating mobile applications.

History of Java

- Java was developed by James Gosling, who is known as the father of Java, in 1995.
- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc.

Following are given significant points that describe the history of Java.

1) **James Gosling, Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

2) Initially it was designed for small, embedded systems in electronic appliances like set-top boxes.

3) Firstly, it was called **"Greentalk"** by James Gosling, and the file extension was .gt.

4) After that, it was called **Oak** and was developed as a part of the Green project.

Why Java was named as "Oak"?

5) **Why Oak?** Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.
6) In 1995, Oak was renamed as **"Java"** because it was already a trademark by Oak Technologies.

Why Java Programming named "Java"?
7) Java is an island in Indonesia where the first coffee was produced (called Java coffee). James Gosling while having a cup of coffee nearby his office chose Java name.
8) Notice that Java is just a name, not an acronym.
9) JDK 1.0 was released on January 23, 1996.

Features of Java

The features of Java are also known as Java buzzwords.

A list of the most important features of the Java language is given below.

1. Simple
2. Object-Oriented
3. Platform independent
4. Secured
5. Robust
6. Interpreted
7. Dynamic

Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystem, Java language is a simple programming language because:

- Java syntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

Object-Oriented (everything is an object)

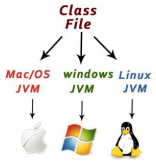
- Java is an object-oriented programming language.
- Everything in Java is an object.
- Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behaviour.
- Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.

Basic concepts of OOPs are:

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

Platform Independent

- Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc.
- Java code is compiled by the compiler and converted into bytecode.
- This bytecode is a platform-independent code because it can be run on multiple platforms,
- i.e., Write Once and Run Anywhere (WORA).



Secured

- Java is best known for its security. With Java, we can develop virus-free systems.

Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**
- **Classloader:** Classloader in Java is a part of the Java Runtime Environment (JRE) which is used to load Java classes into the Java Virtual Machine dynamically.
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access rights to objects.
- **Security Manager:** It determines what resources a class can access such as reading and writing to the local disk.

Robust

- The English mining of Robust is

strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers that avoids security problems.
- Java provides automatic garbage collection which runs on the Java Virtual Machine to get rid of objects which are not being used by a Java application anymore.
- There are exception handling and the type checking mechanism in Java. All these points make Java robust.

Interpreted

- Java is a platform independent programming language
- It means that we can run java on platform that have a java interpreter.
- **To convert the byte code into machine code we deploy the .class file on the JVM.**
- The JVM converts that code into machine code using the java interpreter.

- The compiler compiles the source code into the java bytecode.
- In the same way, the java interpreter converts or translates the bytecode into the machine code.
- After that the machine code interacts with the operating system.

Dynamic

- Java is a dynamic language.
- It supports the dynamic loading of classes.
- It means classes are loaded on demand.
- It also supports functions from its native languages, i.e., C and C++.

Using data within java program

In Java**,** using data means storing, processing, and manipulating values through the use of variables, data types, and operations**.**

Java Variables

- A variable is a container which holds the value while the Java program is executed.
- A variable is assigned with a data type.
- Variable is a name of memory location.
- There are three types of variables in java: local, instance and static.
- There are two types of data types in Java: primitive and non-primitive.

Types of Variables

There are three types of variables in Java:

- local variable
- instance variable
- static variable

1) Local Variable

- Sometimes to meet temporary requirements of the programmers. We can declare variables inside a method or block or constructor such type of variable are called Local variable or temporary variable or stack variable.
- You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.
- A local variable cannot be defined with "static" keyword.
- Local variable will be stored inside stack memory.

2) Instance Variable

- A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.
- We can’t access instance variable directly from static area but we can access by using object reference.
- It is called an instance variable because its value is instance-specific and is not shared among instances.
- Instance variable will be stored in the Heap memory at part of object.

3) Static variable

- A variable that is declared as static is called a static variable. It cannot be local.
- Static variable should be declare within the class directly but outside of any method or block of constructor.
- We can access static variable either by object reference or by class name but recommended to use class name.
- Memory allocation for static variables happens only once when the class is loaded in the memory.
- Static variable will be stored in Method area.

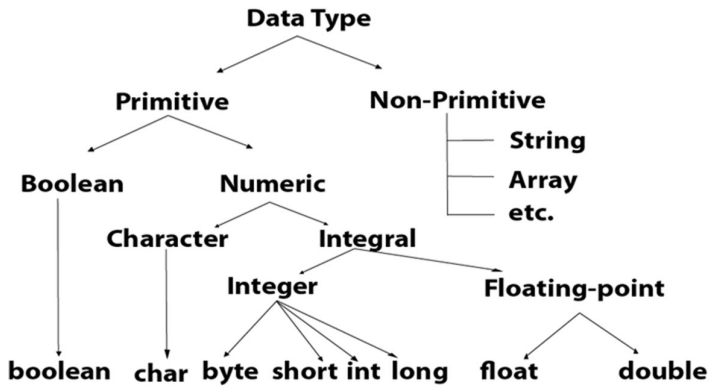
Example to understand the types of variables in java

```
1.  public class A
2.  {
3.      static int m=100;//static
        variable
4.      void method()
6.        int n=90;//local
7.      }
8.      public static void main(String
        args[])
10.       int data=50;//instance
11.     }
12. }//end of
```

Data Types in Java

- Data types specify the different sizes and values that can be stored in the variable.

There are two types of data types in Java:



1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes,String, Interfaces, and Arrays.

There are 8 types of primitive data types:

- boolean data type
- byte data type
- char data type
- short data type
- int data type
- long data type
- float data type
- double data type

Data Type	Default Value	Default size
boolean	false	1 bit
Char	'\u0000'	2 byte
Byte	0	1 byte
Short	0	2 byte
Int	0	4 byte
Long	0L	8 byte
Float	0.0f	4 byte
Double	0.0d	8 byte

Boolean Data Type

- The Boolean data type is used to store only two possible values: true and false.
- This data type is used for simple flags that track true/false conditions.

The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.

Example:

1. Boolean one = **false**

Byte Data Type

- The byte data type is an example of primitive data type.
- It is an 8-bit signed two's complement integer.
- Its value-range lies between -128 to 127 (inclusive).
- Its minimum value is -128 and maximum value is 127.
- Its default value is 0.

- The byte data type is used to save memory in large arrays where the memory savings is most required.
- It saves space because a byte is 4 times smaller than an integer.
- It can also be used in place of "int" data type.

Example:

1. **byte** a = 10, **byte** b = -20

Short Data Type

- The short data type is a 16-bit signed two's complement integer.
- Its value-range lies between -32,768 to 32,767 (inclusive).
- Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.
- The short data type can also be used to save memory just like byte data type.
- A short data type is 2 times smaller than an integer.

Example:

1. **short** s = 10000, **short** r = -5000

Int Data Type

- The int data type is a 32-bit signed two's complement integer.
- Its value-range lies between - 2,147,483,648 (-2^31) to 2,147,483,647 (2^31 -1) (inclusive).
- Its minimum value is - 2,147,483,648and maximum value is 2,147,483,647.
- Its default value is 0.

The int data type is generally used as a default data type for integral values unless if there is no problem about memory.

Example:

1. **int** a = 100000, **int** b = -200000

Long Data Type

- The long data type is a 64-bit two's complement integer.
- Its value-range lies between -9,223,372,036,854,775,808(-2^63) to 9,223,372,036,854,775,807(2^63 -1)(inclusive).
- Its minimum value is - 9,223,372,036,854,775,808and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Example:

1. **long** a = 100000L, **long** b = -200000L

Float Data Type

- The float data type is a single-precision 32-bit IEEE 754 floating point.
- -3.4e38 to 3.4e38
- It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Example:

1. **float** f1 = 234.5f

Double Data Type

- The double data type is a double-precision 64-bit IEEE 754 floating point.
- -1.7r308 to 1.7e308
- The double data type is generally used for decimal values just like float.
- The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Example:

1. **double** d1 = 12.3

Char Data Type

- The char data type is a single 16-bit Unicode character.
- Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive).
- The char data type is used to store characters.

Example:

1. **char** letterA = 'A'

Understanding numeric, Type Conversion

- In Java, **type casting** is a method or process that converts a data type into another data type in both ways manually and automatically.
- The compiler and manual conversion performed by the programmer do the automatic conversion.
- Numeric type conversion in Java is an essential concept, particularly when dealing with different numeric data types.
- Java provides both implicit and explicit ways to convert between these types.

Numeric Data Types in Java

1. byte: 8-bit integer
2. short: 16-bit integer
3. int: 32-bit integer
4. long: 64-bit integer
5. float: 32-bit floating-point
6. double: 64-bit floating-point

Implicit Conversion (Widening Conversion)

- Converting a lower data type into a higher one is called **implicit conversion** type casting.
- It is also known as **widening** or **casting down**.
- It is done automatically.
- It is safe because there is no chance to lose data.
- Implicit conversions happen automatically when converting a smaller type to a larger type (widening conversion).
- This is safe and doesn't result in data loss.

- **byte** -> **short** -> **int** -> **long** -> **float** -> **double**

Example:

```
public class Main {
   public static void main(String[] args) {
      int a = 100;
      double b = a; // int to double conversion
      System.out.println("int value: " + a);  // 100
      System.out.println("double value: " + b); // 100.0
   }
}
```

Explicit Conversion (Narrowing Conversion)

- Converting a higher data type into a lower one is called **explicit conversion** type casting.
- It is also known as **narrowing** or **casting up**.
- It is done manually by the programmer.]
- If we do not perform casting then the compiler reports a compile-time error.
- To perform explicit conversion, you use the cast operator (type).

- **double** -> **float** -> **long** -> **int** -> **short** -> **byte**

Example:

```
public class Main {
   public static void main(String[] args) {
      double a = 100.04;
      int b = (int) a; // explicit conversion from double to int
      System.out.println("double value: " + a);  // 100.04
      System.out.println("int value: " + b); // 100 (fractional part is truncated)
   }
}
```

Type Conversion Methods

- Java provides wrapper classes for each of the primitive types (e.g., Integer, `Double`) that include methods for converting strings to numeric types and vice versa.

- String to int:

```
 String str = "123";

int num = Integer.parseInt(str);
```

- String to double:

```
 String str = "123.45";

double num = Double.parseDouble(str);
```

- int to String:

```
 int num = 123;

String str = Integer.toString(num);
```

- double to String:

```
 double num = 123.45;
String str = Double.toString(num);
```

Operators in Java

- **Operator** in Java is a symbol that is used to perform operations.
- For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

- Unary Operator,
- Arithmetic Operator,
- Shift Operator,
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

Java Operator Precedence

Operator Type	Category	Precedence
Unary	postfix	*expr*++ *expr*--
	prefix	++*expr* --*expr* +*expr* -*expr* ~ !
Arithmetic	multiplicative	* / %
	additive	+ -
Shift	shift	<< >> >>>
Relational	comparison	< > <= >= instanceof
	equality	== !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Java Unary Operator

- The Java unary operators require only one operand.
- Unary operators are used to perform various operations i.e.
  - incrementing/decrementing a value by one
  - negating an expression
  - inverting the value of a Boolean

Java Unary Operator

Example: ++ and --

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int x=10;
4. System.out.println(x++);//10  (11)
5. System.out.println(++x);//12
6. System.out.println(x--);//12  (11)
7. System.out.println(--x);//10
8. }}
```

Output:

10
12
12
10

Java Unary Operator

Example 2: ++ and --

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=10;
5. System.out.println(a++ + ++a);//10+12=22
6. System.out.println(b++ + b++);//10+11=21
7.
8. }}
```

Output:

22

21

Java Unary Operator Example: ~ and !

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=-10;
5. boolean c=true;
6. boolean d=false;
7. System.out.println(~a);//-11 (minus of total positive value which starts from 0)
8. System.out.println(~b);//9 (positive of total minus, positive starts from 0)
9. System.out.println(!c);//false (opposite of boolean value)
10. System.out.println(!d);//true
11. }}
```

Output:

-11
9
false
true

Java Arithmetic Operators

- Java arithmetic operators are used to perform addition, subtraction, multiplication, and division.
- They act as basic mathematical operations.

Java Arithmetic Operator Example

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=5;
5. System.out.println(a+b);//15
6. System.out.println(a-b);//5
7. System.out.println(a*b);//50
8. System.out.println(a/b);//2
9. System.out.println(a%b);//0
10. }}
```

Output:

15
5
50
2
0

Java Arithmetic Operator Example: Expression

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. System.out.println(10*10/5+3-1*4/2);
4. }}
```

Output:

21

Java Left Shift Operator

- The Java left shift operator << is used to shift all of the bits in a value to the left side of a specified number of times.

Java Left Shift Operator Example:

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. System.out.println(10<<2);//10*2^2=10*4=40
4. System.out.println(10<<3);//10*2^3=10*8=80
5. System.out.println(20<<2);//20*2^2=20*4=80
6. System.out.println(15<<4);//15*2^4=15*16=240
7. }}
```

Output:

40
80
80
240

Java Right Shift Operator

- The Java right shift operator >> is used to move the value of the left operand to right by the number of bits specified by the right operand.

Java Right Shift Operator Example:

```
1. public OperatorExample{
2. public static void main(String args[]){
3. System.out.println(10>>2);//10/2^2=10/4=2
4. System.out.println(20>>2);//20/2^2=20/4=5
5. System.out.println(20>>3);//20/2^3=20/8=2
6. }}
```

Output:

2

5

2

Java Shift Operator Example: >> vs >>>

```
1. public class OperatorExample{
2. public static void main(String args[]){
3.     //For positive number, >> and >>> works same
4.     System.out.println(20>>2);
5.     System.out.println(20>>>2);
6.     //For negative number, >>> changes parity bit (MSB(most significant bit.) to 0
7.     System.out.println(-20>>2);
8.     System.out.println(-20>>>2);
9. }}
```

Output:

5

5

-5

1073741819

<u>Java AND Operator Example: Logical && and Bitwise &</u>

- The logical && operator doesn't check the second condition if the first condition is false.
- It checks the second condition only if the first one is true.

- The bitwise & operator always checks both conditions whether first condition is true or false.

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=5;
5. int c=20;
6. System.out.println(a<b&&a<c);//false && true = false
7. System.out.println(a<b&a<c);//false & true = false
8. }}
```

Output:

false

false

Java AND Operator Example: Logical && vs Bitwise &

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=5;
5. int c=20;
6. System.out.println(a<b&&a++<c);//false && true = false
7. System.out.println(a);//10 because second condition is not checked
8. System.out.println(a<b&a++<c);//false && true = false
9. System.out.println(a);//11 because second condition is checked
10. }}
```

Output:

false
10
false
11

Java OR Operator Example: Logical || and Bitwise |

- The logical || operator doesn't check the second condition if the first condition is true.
- It checks the second condition only if the first one is false.

- The bitwise | operator always checks both conditions whether first condition is true or false.

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=5;
5. int c=20;
6. System.out.println(a>b||a<c);//true || true = true
7. System.out.println(a>b|a<c);//true | true = true
8. //|| vs |
9. System.out.println(a>b||a++<c);//true || true = true
10. System.out.println(a);//10 because second condition is not checked
11. System.out.println(a>b|a++<c);//true | true = true
12. System.out.println(a);//11 because second condition is checked
13. }}
```

Output:

true
true
true
10
true
11

Java Ternary Operator

- Java Ternary operator is used as one line replacement for if-then-else statement and used a lot in Java programming.
- It is the only conditional operator which takes three operands.

Java Ternary Operator Example:

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=2;
4. int b=5;
5. int min=(a<b)?a:b;
6. System.out.println(min);
7. }}
```

Output:

2

Java Assignment Operator

- Java assignment operator is one of the most common operators.
- It is used to assign the value on its right to the operand on its left.

Java Assignment Operator Example:

```
1. public class OperatorExample{
2. public static void main(String args[]){
3. int a=10;
4. int b=20;
5. a+=4;//a=a+4 (a=10+4)
6. b-=4;//b=b-4 (b=20-4)
7. System.out.println(a);
8. System.out.println(b);
9. }}
```

Output:

14
16

Q. What is JVM, JRE, and JDk? JVM

JVM (Java Virtual Machine) is an abstract machine. It is called a virtual machine because it doesn't physically exist. It is a specification that provides a runtime environment in which Java bytecode can be executed. It can also run those programs which are written in other languages and compiled to Java bytecode.

JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent. There are three notions of the JVM: specification, implementation, and instance.

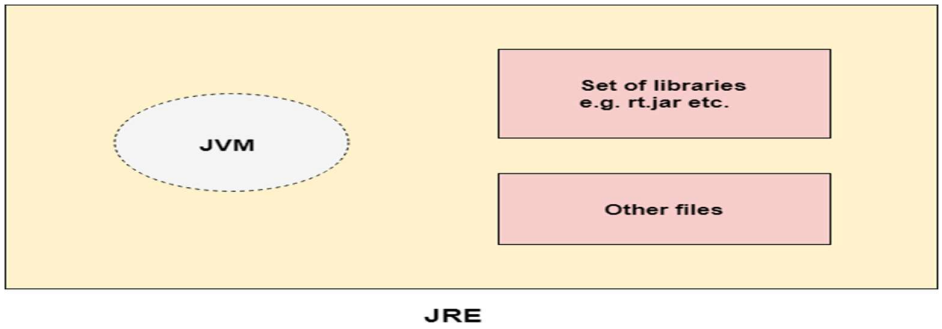
The JVM performs the following main tasks:

- Loads code
- Verifies code
- Executes code
- Provides runtime environment

JRE

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.

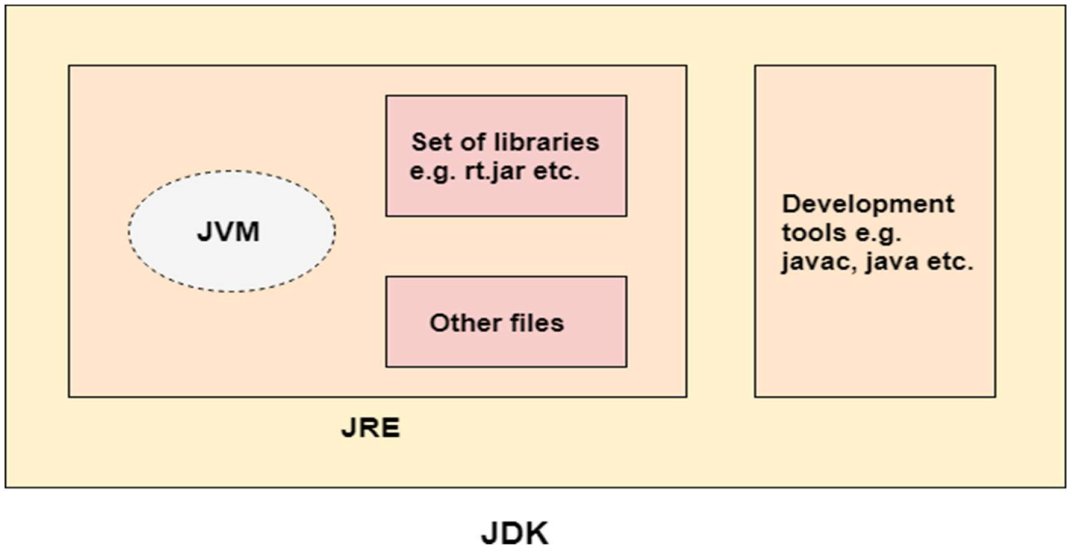
The implementation of JVM is also actively released by other companies besides Sun Micro Systems.



JDK

JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools.

JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:

- Standard Edition Java Platform
- Enterprise Edition Java Platform
- Micro Edition Java Platform

The JDK contains a private Java Virtual Machine (JVM) and a few other resources such as an interpreter/loader (java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc), etc. to complete the development of a Java Application.